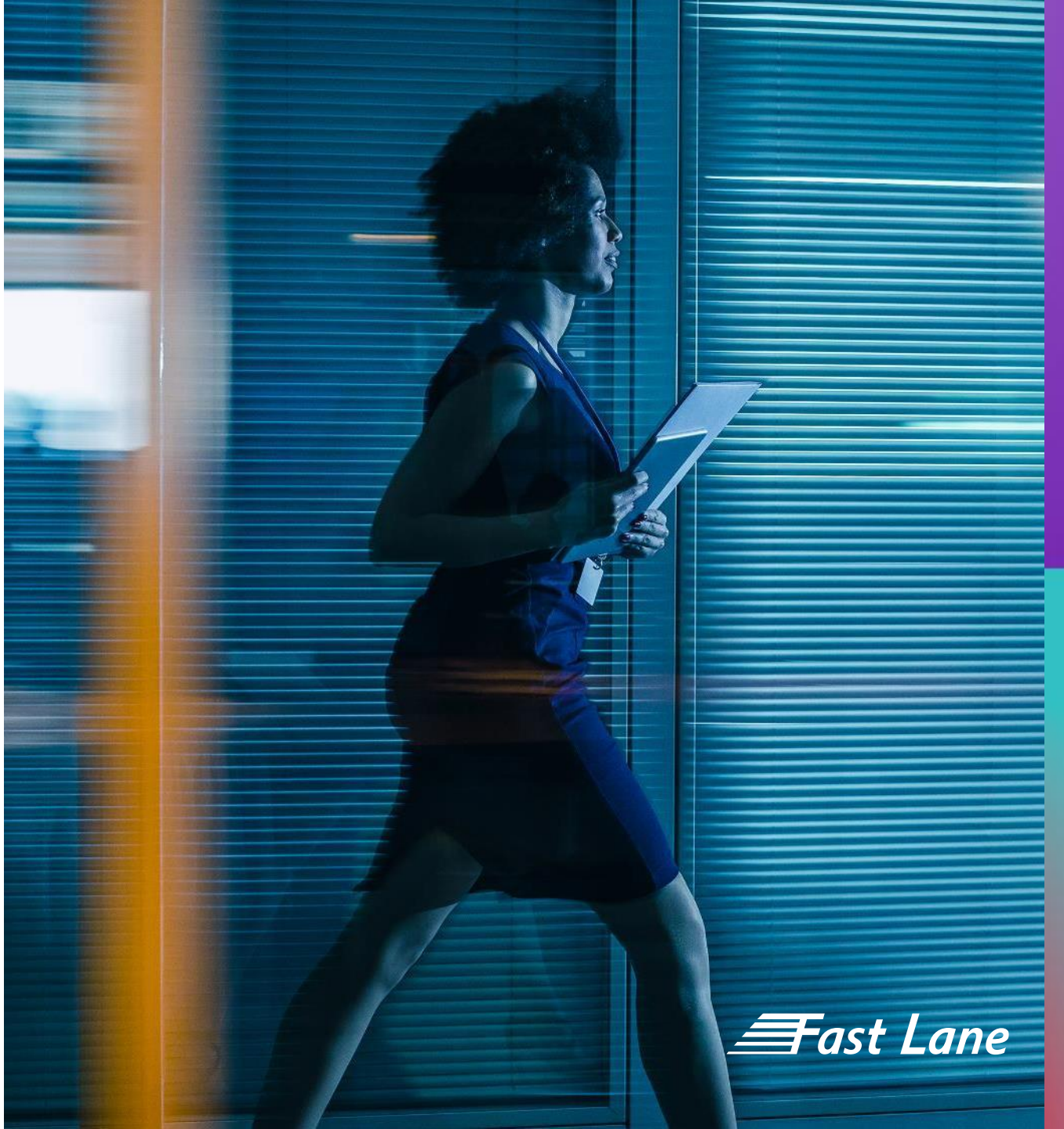
Fast Lane Group

Worldwide Education & Professional Services

Microsoft Solutions Partner

Microsoft Cloud Training Services



www.flane.de | © 2026 Fast Lane Institute for Knowledge Transfer GmbH

— AGENTIC AI: FROM STRATEGY TO **PRODUCTION.**

Alexander Kastil

Full stack, from enterprise software to agentic AI. Since 1996

AI at Work
Value discovery, KPI mapping and roadmap

Enterprise AI
Governed multi-agent systems on Microsoft Copilot and Foundry

Agentic Software Engineering
Harness architecture for Claude Code and GitHub Copilot

Team Enablement
AI driven content production and delivery

integrations.at
alexander.kastil@integrations.at

github.com/alexander-kastil
linkedin.com/in/alexander-kastil

INTEGRATIONS

AGENTIC AI CONSULTING · VIENNA



Microsoft CERTIFIED — AGENTIC AI BUSINESS SOLUTIONS ARCHITECT — ★★★

Microsoft CERTIFIED — AZURE AI APP AND AGENT DEVELOPER — ASSOCIATE ★★

Microsoft CERTIFIED — DEVOPS ENGINEER — EXPERT ★★★

SCAN TO CONNECT
integrations.at

MODULE 1

Claude Code Fundamentals

From first install to parallel background sessions

Agentic Software Engineering with Claude Code



01

Get Started

Product family, install, sign-in and editors

02

Model Selection

Fable, Opus, Sonnet, Haiku and a warm prompt cache

03

Permissions

Tiers, rules, modes and settings scopes

04

Checkpoints and Worktrees

Rewind a session, isolate parallel work

05

Agents View

Dispatch and triage background sessions

06

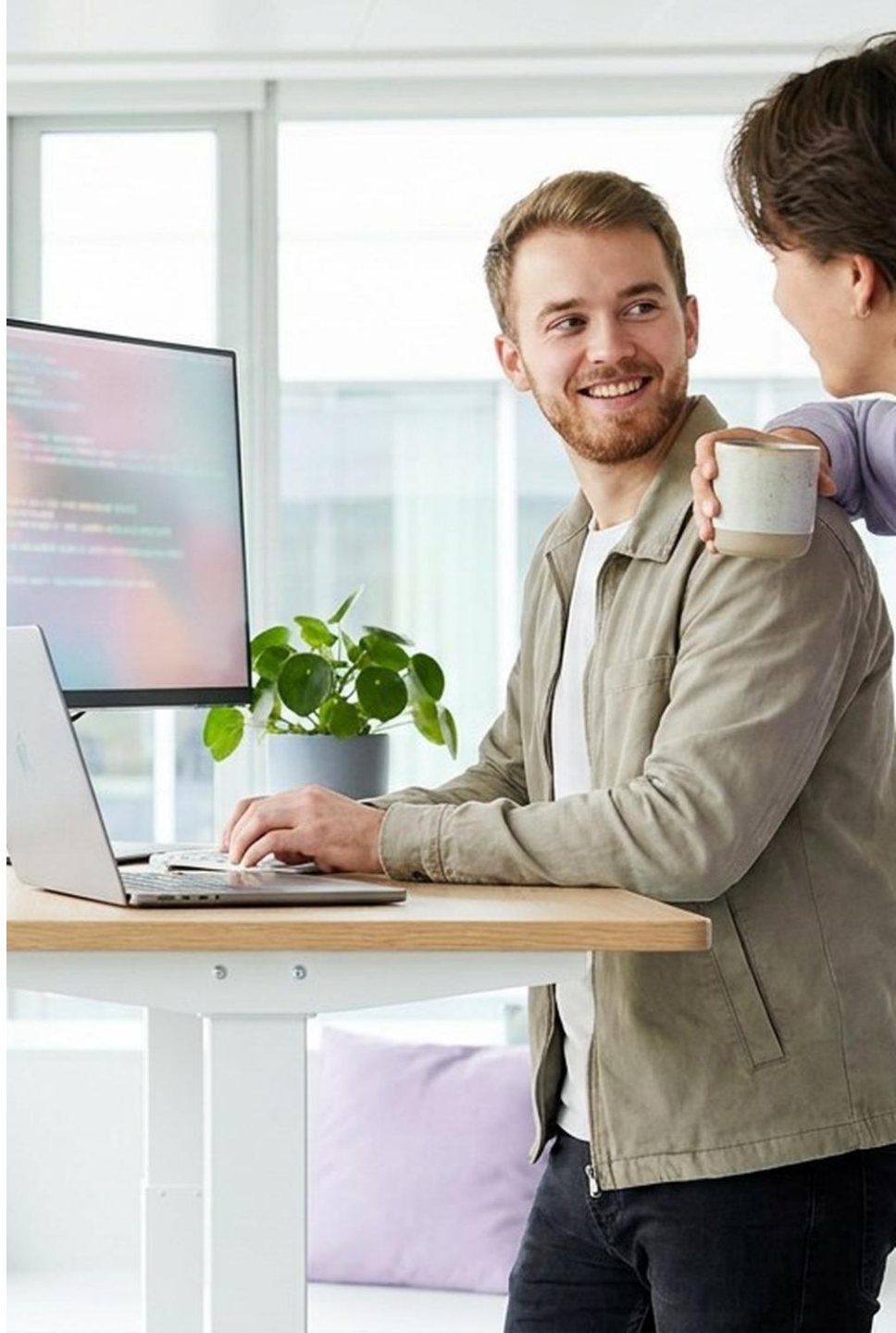
Artifacts

Publish, version and share live pages

From first install to parallel background sessions

Claude Code fundamentals

- **Getting started**: product family, install, sign-in and the editor that fits
- **Cost and control**: model choice, the prompt cache and permission rules
- **Parallel work**: checkpoints, worktrees, the agents view and artifacts



The path through the fundamentals

Overview

- Start with the product family, then install, sign in and pick an editor
- Choose a model by task and keep the **prompt cache** warm
- Set **permissions** before Claude acts, then rewind with checkpoints
- Run parallel work in **worktrees**, supervise it in the agents view
- Finish by publishing results as **artifacts**



01

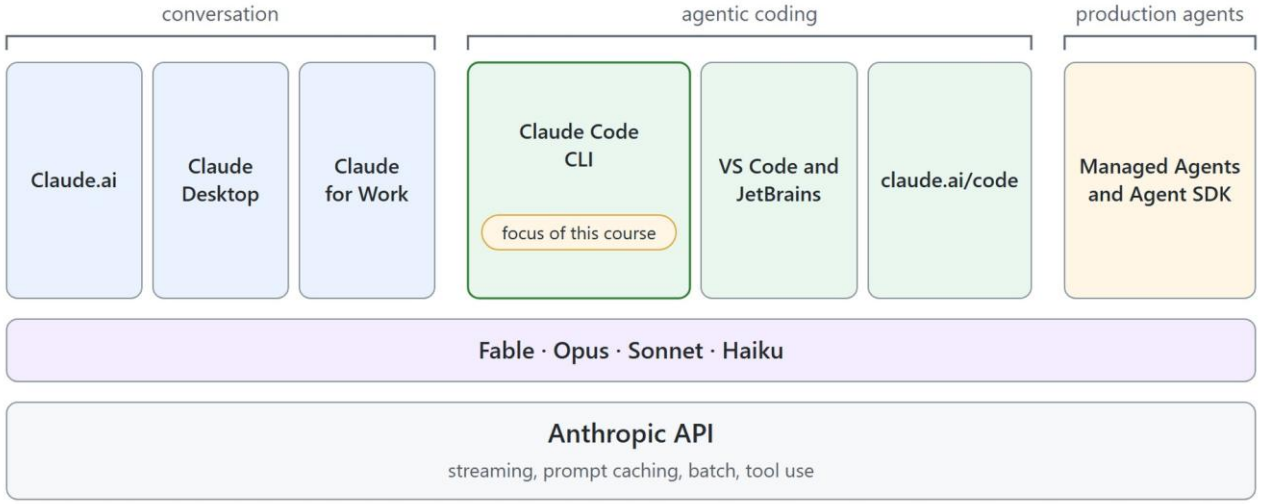
Get Started

Product family, install, sign-in and editors

One API, one model family, many surfaces

The Claude product family

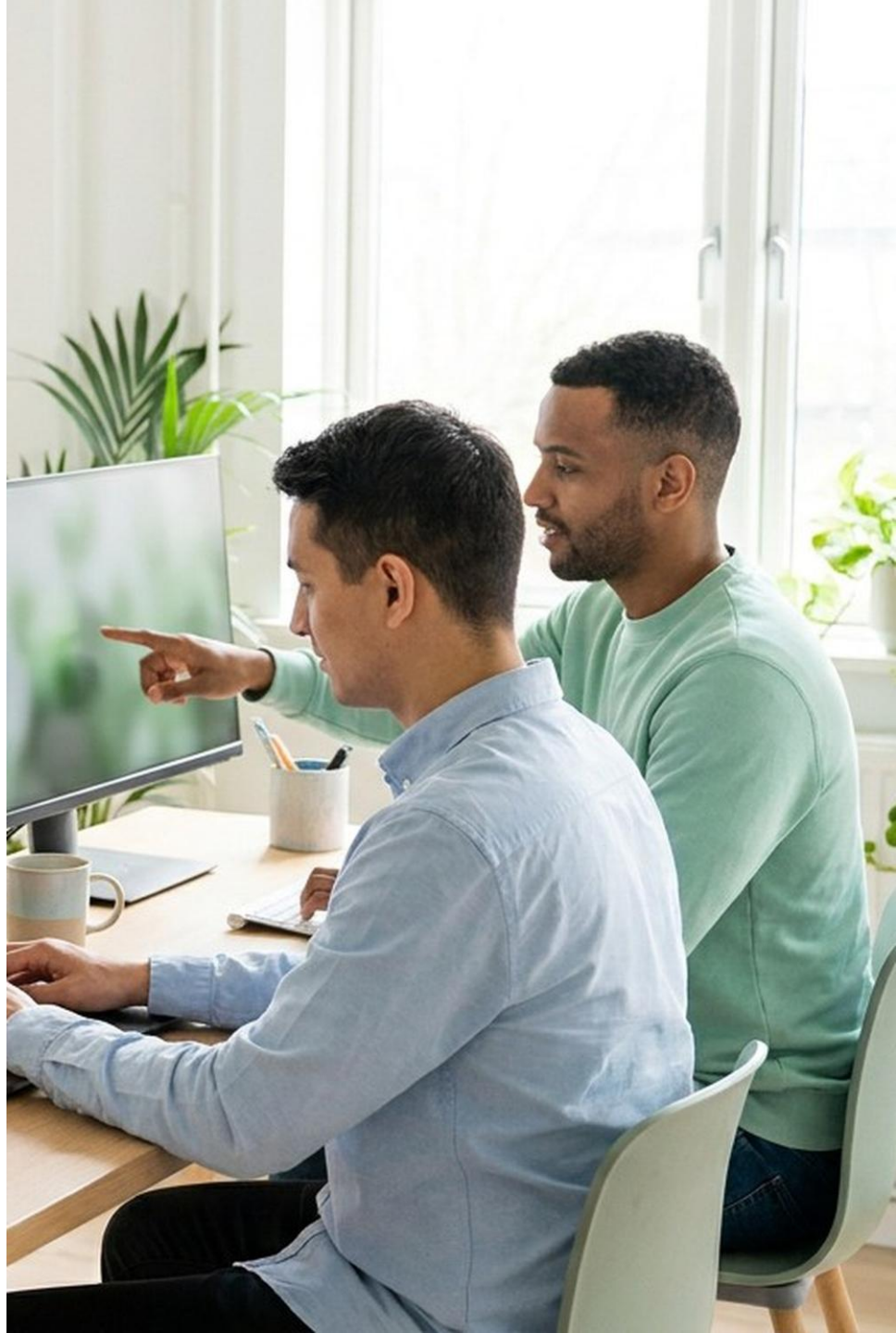
- **Claude.ai** and **Claude Desktop**: conversation, plus local files, MCP and Claude Code on the desktop
- **Claude Code**: CLI, VS Code and JetBrains extensions, and claude.ai/code in the browser
- **Managed Agents** and the **Agent SDK**: Claude as a component inside your own system
- All share Fable, Opus, Sonnet and Haiku, all run on the **Anthropic API**



It runs where the code lives

Why Claude Code for developers

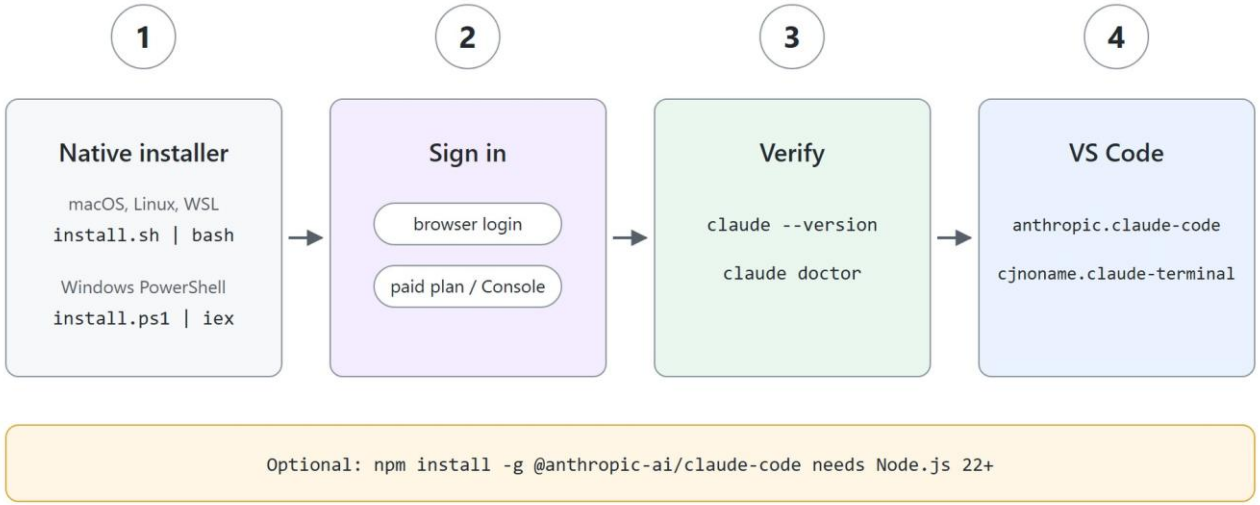
- Runs in the **terminal**, the **IDE** and **CI/CD** pipelines
- Persistent access to the full repository, shell commands and git history
- Multi-file refactors, test-driven bug fixes and whole features from one prompt
- `claude` for a session, `claude "<task>"` for a one-shot run



The native installer, sign-in and a first check

Installing Claude Code

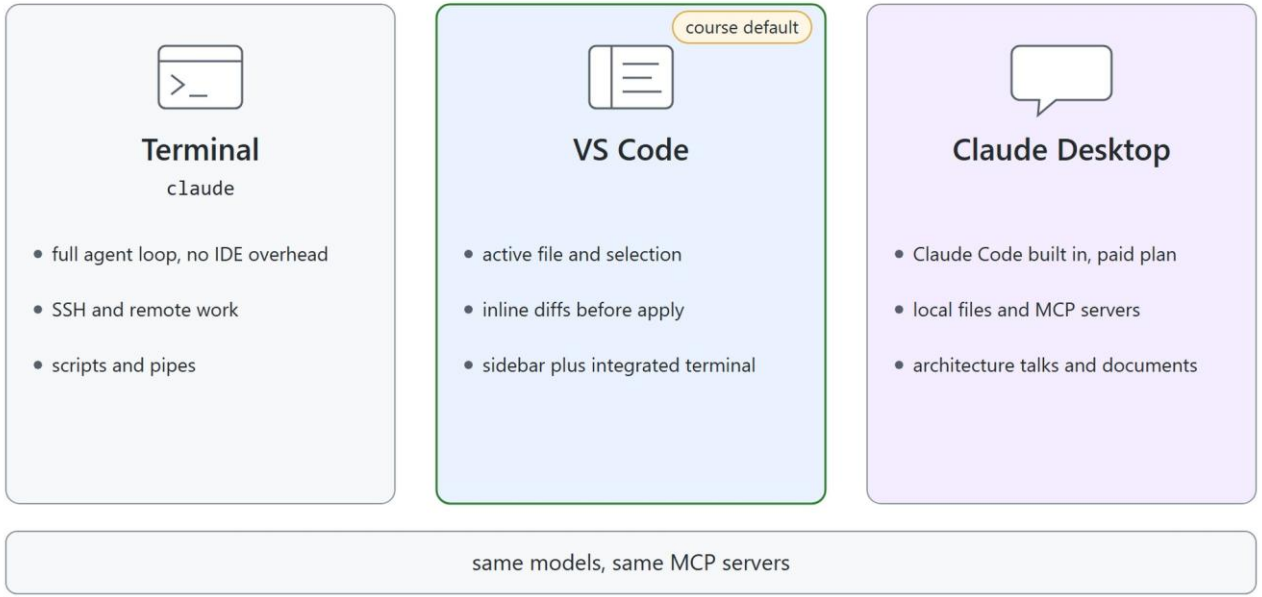
- **Native installer** is the recommended path and updates itself
- `claude.ai/install.sh` on macOS and Linux, install.ps1 on Windows
- First `claude` run: browser login with a paid plan or a Console account
- Run `claude doctor` to check the install
- VS Code: the **Claude Code** and **Claude Terminal** extensions



Which environment fits which working style

Terminal, VS Code and Claude Desktop

- **Terminal**: the native home, no IDE overhead, best for SSH and scripted runs
- **VS Code**: the course default, with active file, selection and inline diffs
- **Claude Desktop**: local files and MCP, with Claude Code built in on a paid plan
- Switch models mid-session with `/model`, check the setup with `/status`



02

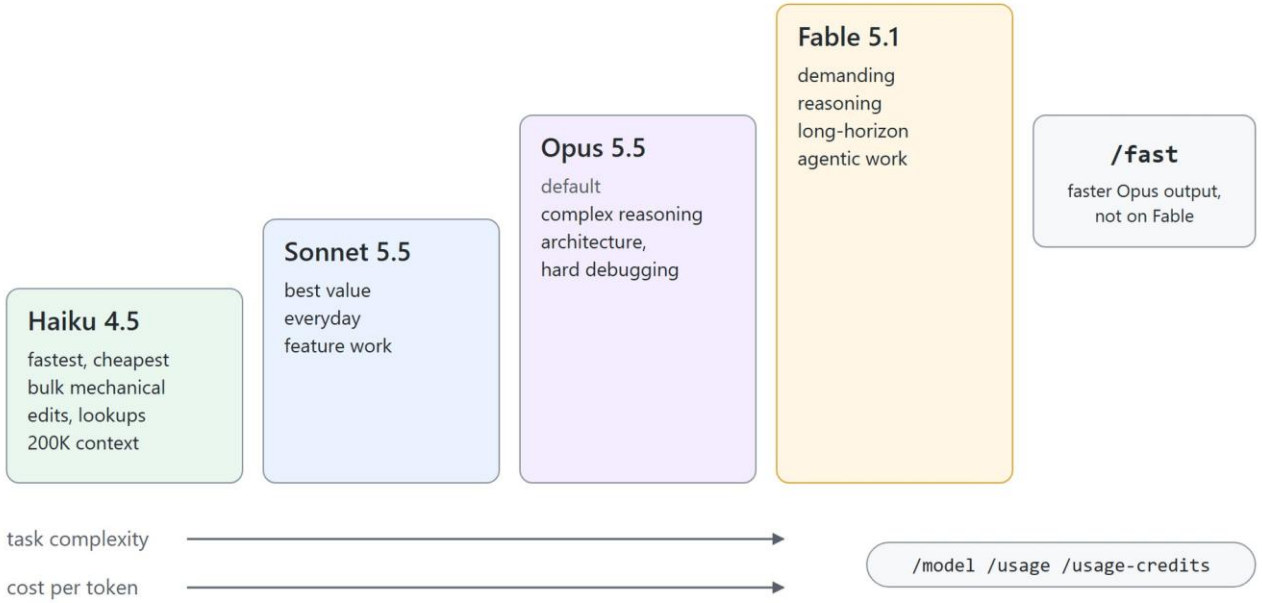
Model Selection

Fable, Opus, Sonnet, Haiku and a warm prompt cache

Match model strength to task complexity

Choosing a model

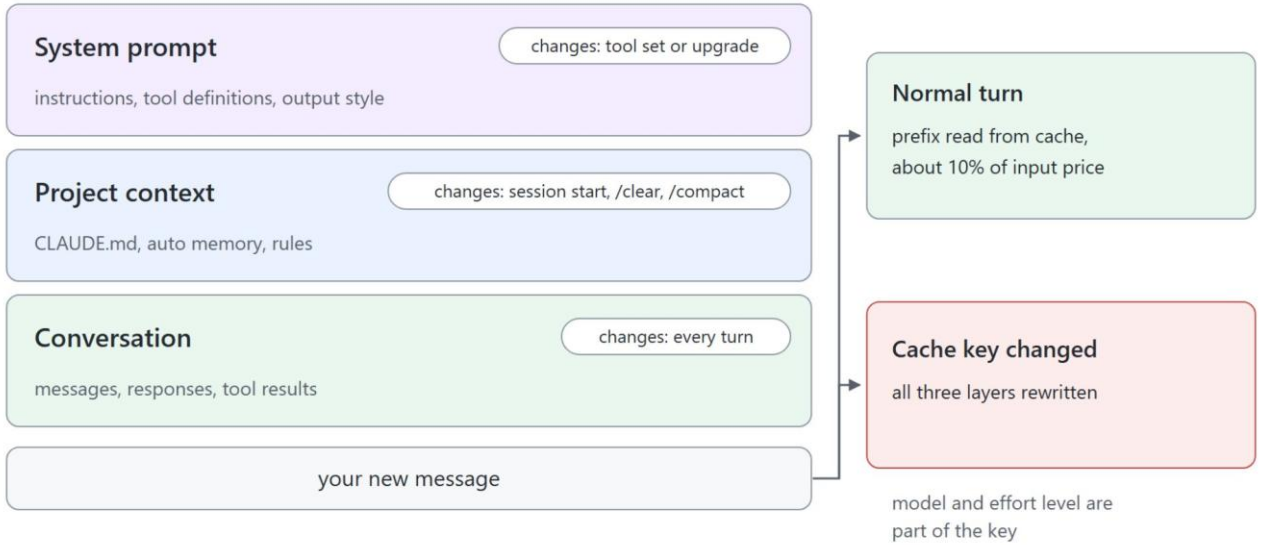
- **Haiku 4.5** for bulk mechanical work, **Sonnet 5.5** for everyday features, **Opus 5.5** for hard reasoning, Fable 5.1 for long-horizon agentic work
- Route by task complexity, not by file count
- `/fast` runs Opus with faster output at a higher price, not on Fable
- `/usage` shows where you stand; `/usage-credits` keeps you working past the plan limit



Every turn re-sends the whole context, the prefix is read from cache

How the prompt cache is organized

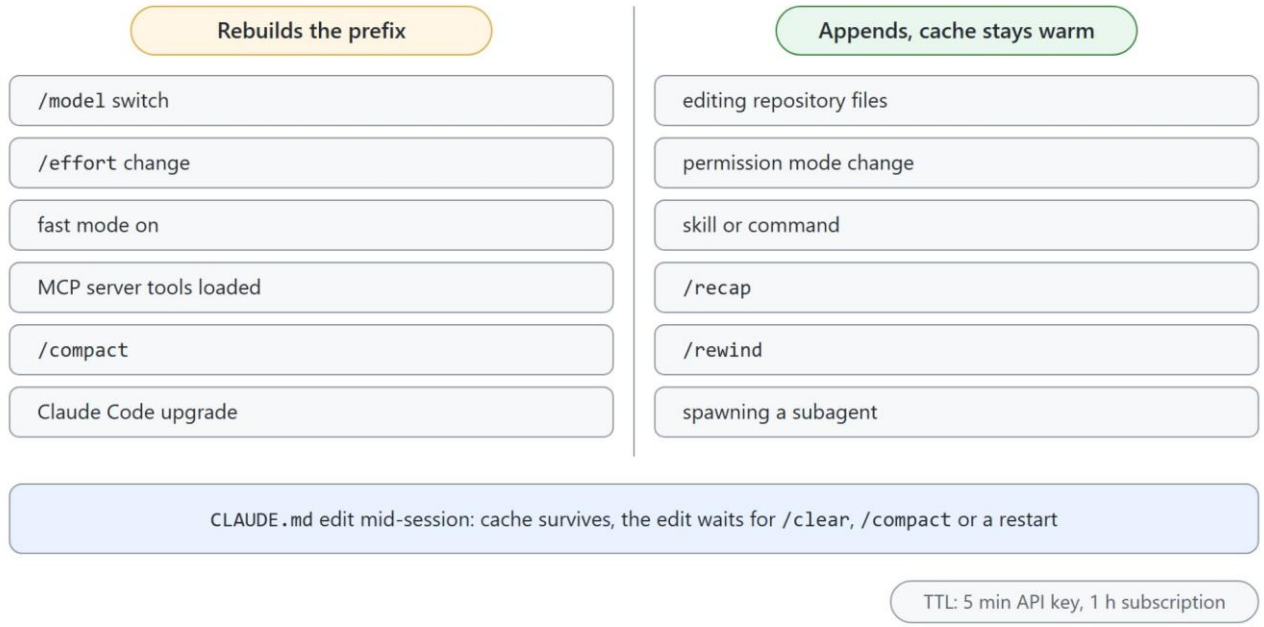
- Three layers: **system prompt**, **project context**, **conversation**
- The cache matches the **prefix** exactly; a change recomputes everything after it
- Cache reads cost about 10% of the input price, cache writes cost more
- Model and effort level are part of the cache key
- Read `cache read` against `cache write` in `/usage`



Know which turn will be slow and expensive

Actions that break or keep the cache

- **Rebuild**: `/model`, `/effort`, fast mode, loaded MCP tools, `/compact`, an upgrade
- **Keep**: file edits, permission modes, skills, `/recap`, `/rewind`, subagents
- A mid-session **CLAUDE.md** edit is silently inert until `/clear` or a restart
- TTL: one hour on a subscription, five minutes on an API key
- Scope: one machine and one directory, worktrees included



03

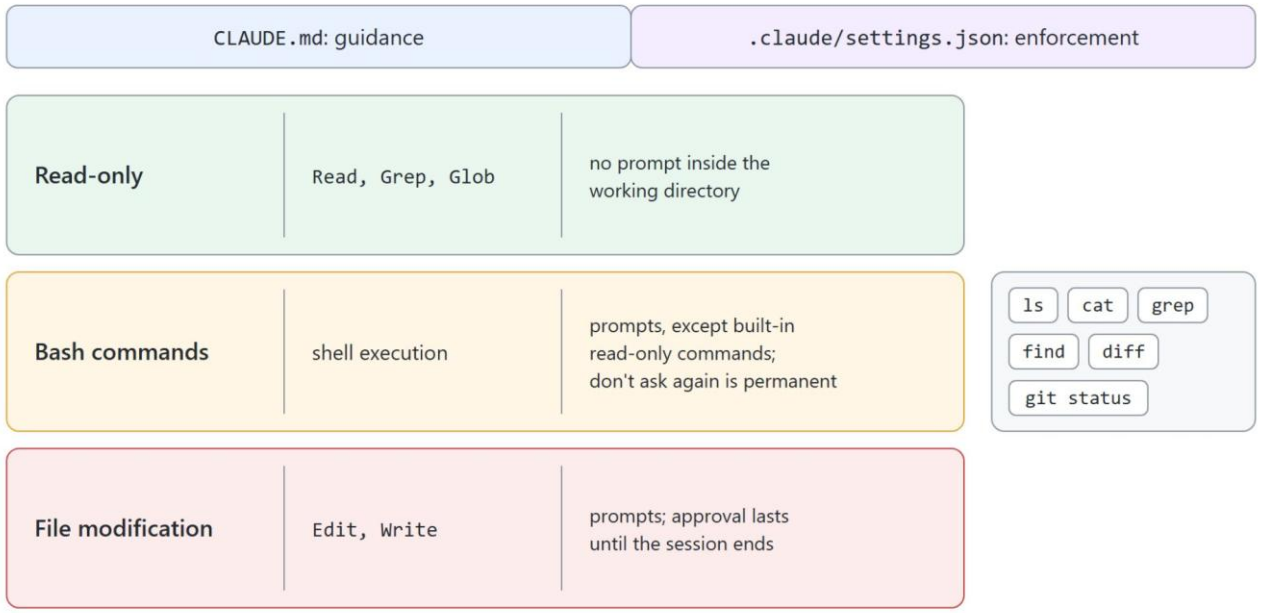
Permissions

Tiers, rules, modes and settings scopes

Enforced by Claude Code, not by the model

Permission tiers

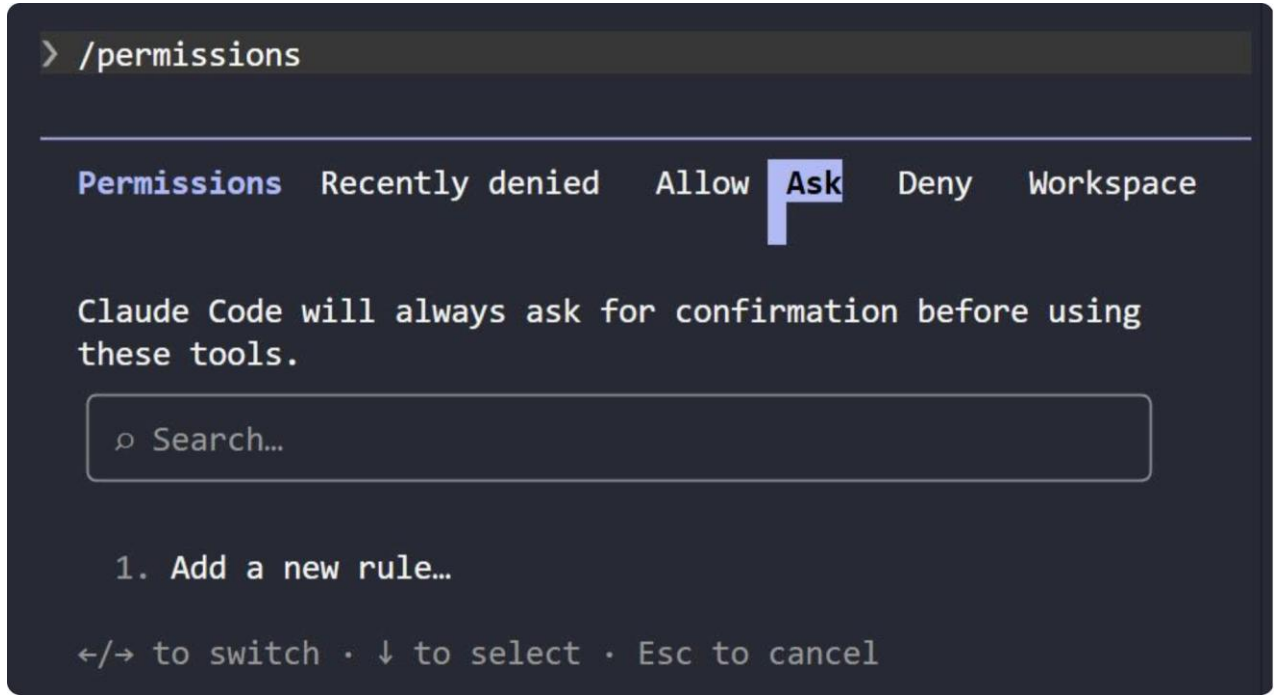
- **Read-only**: file reads, Grep and Glob run without a prompt
- **Bash**: prompts, except a fixed set of read-only commands such as `ls` and `git status`
- **File modification**: Edit and Write prompt, approval lasts for the session
- `CLAUDE.md` is guidance, `.claude/settings.json` is enforcement



Writing permission rules

Deny, then ask, then allow

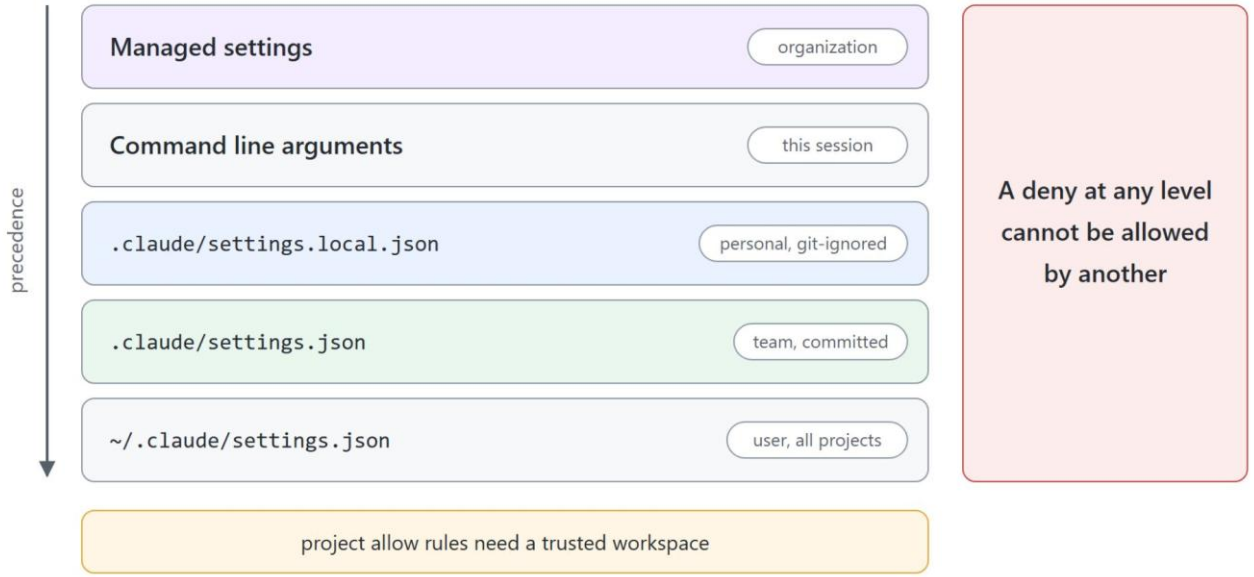
- Rules take the form `Tool` or `Tool(specifier)` in **allow**, **ask** and **deny** lists
- First match wins in the order **deny, ask, allow**, so a deny takes no exceptions
- `Bash(ls *)` keeps a word boundary; compound commands are checked per subcommand
- Denying bare `Bash` hides the tool; `Bash(rm *)` blocks only matching calls
- `/permissions` lists every rule and the file it came from



Settings scopes and precedence

Which file decides, and why a deny always holds

- **Managed** settings outrank command line, local, project and user settings
- Commit team policy in `.claude/settings.json`, keep personal rules in `settings.local.json`
- A deny at any level cannot be allowed by another level
- Project allow rules apply only in a **trusted** workspace



04

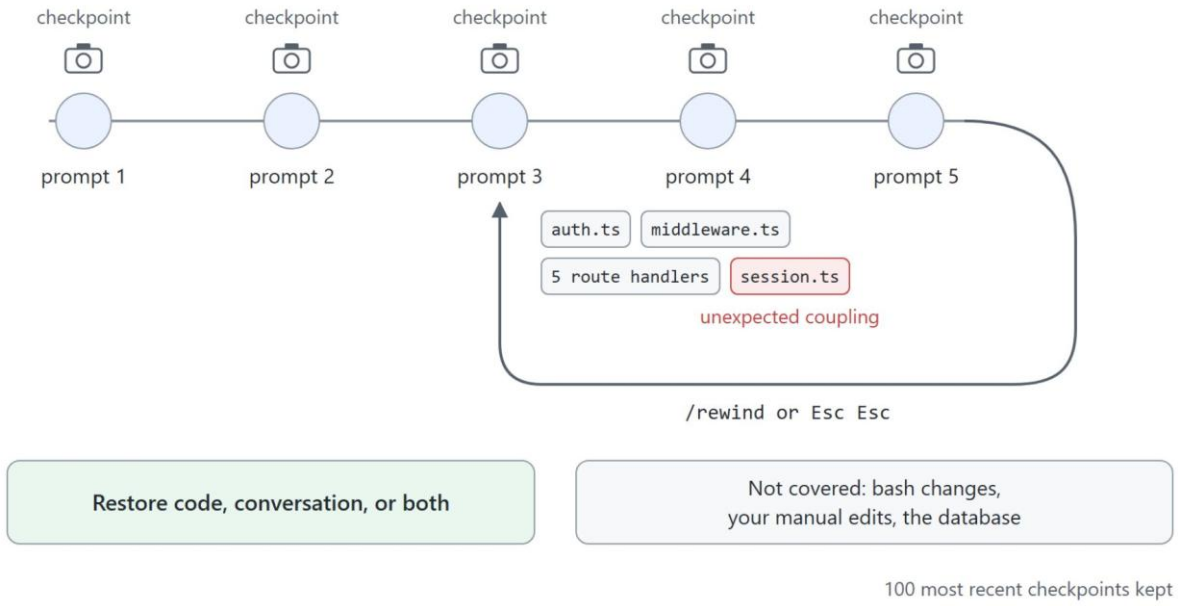
Checkpoints and Worktrees

Rewind a session, isolate parallel work

Checkpoints and rewind

Undo a runaway refactor in one step

- A **checkpoint** is taken automatically before every prompt you send
- `/rewind` or `Esc` twice: restore code, conversation, or both
- Covers files Claude edited with its own tools, never bash changes or the database
- Ask for an explicit checkpoint before anything risky



Parallel sessions with git worktrees

One repository, an isolated checkout per session

- A **worktree** is a separate checkout on its own branch, sharing one `.git` store
- `claude --worktree <name>` creates `.claude/worktrees/<name>` on branch `worktree-<name>`
- Background sessions and isolated subagents enter a worktree on their own
- `.worktreeinclude` copies `.env` and other git-ignored files into each one
- Cleanup is two steps: remove the worktree, then delete its branch



05

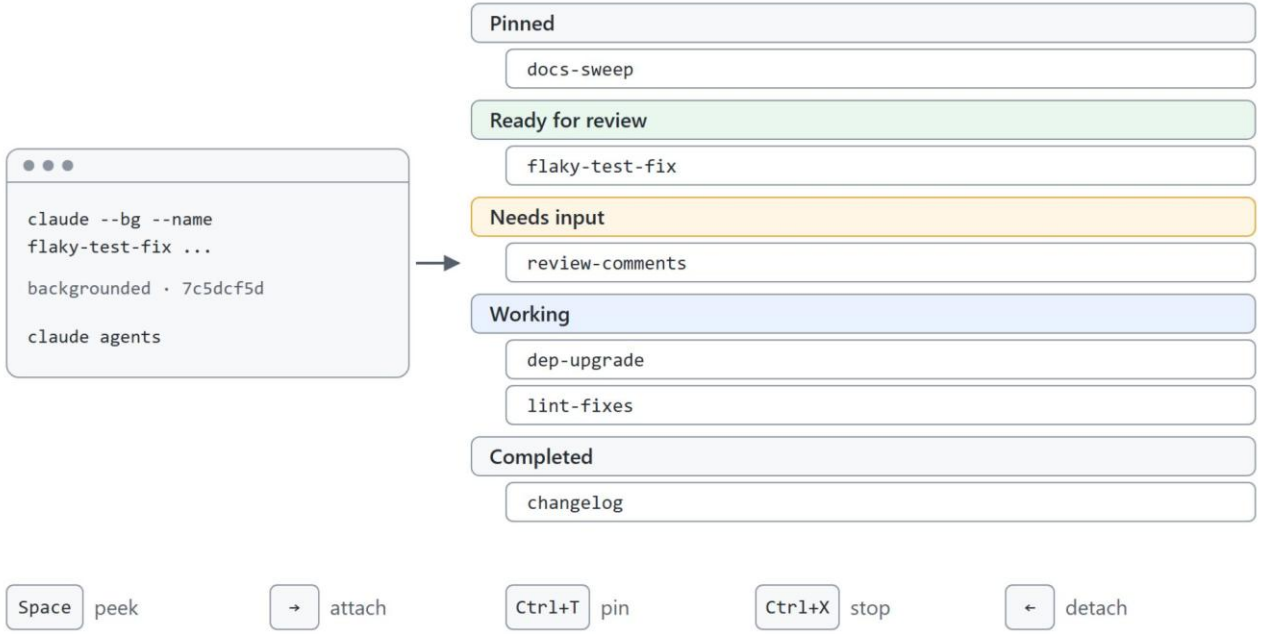
Agents View

Dispatch and triage background sessions

The agents view

Dispatch background sessions and triage them from one board

- `claude --bg "<task>"` returns at once; `/background` sends the current session
- `claude agents` groups sessions: pinned, **ready for review**, **needs input**, working, completed
- `Space` peeks and replies without attaching; detaching never stops a session
- Each background session edits in its own worktree under `.claude/worktrees/`



Supervising a fleet of sessions

The supervisor, and the cost of fanning out

- A per-user **supervisor** keeps sessions alive across terminals and sleep
- Idle sessions stop after about an hour unless pinned with `Ctrl+T`
- `claude daemon status` and `~/.claude/daemon.log` when something is stuck
- Ten parallel agents use about **ten times** the quota of one



06

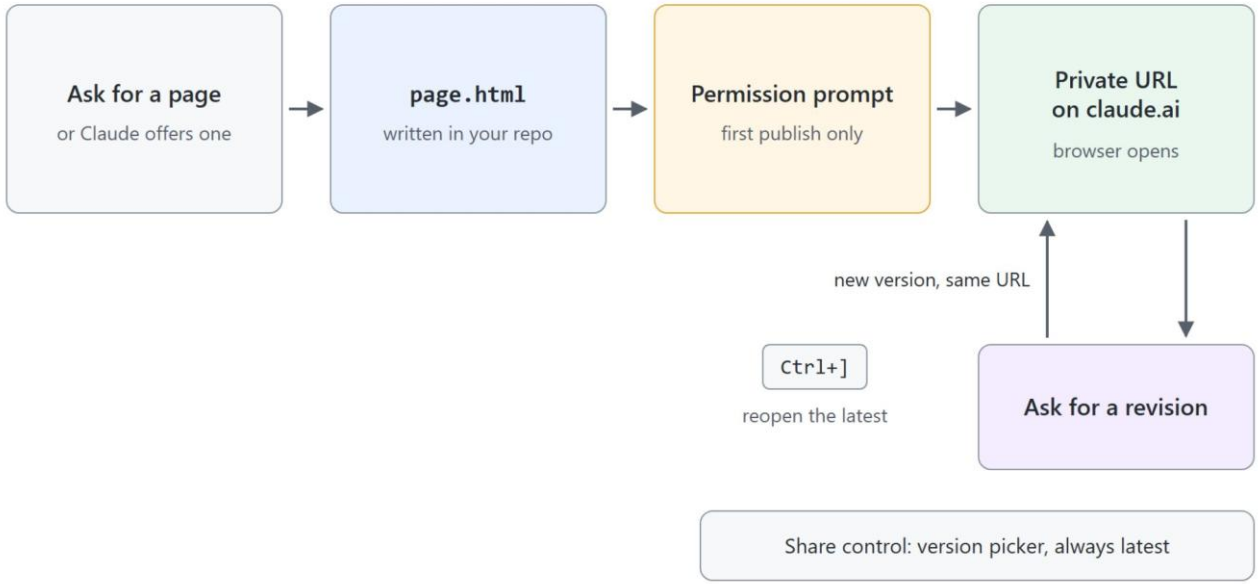
Artifacts

Publish, version and share live pages

Publishing an artifact

Session output as a live page on claude.ai

- An **artifact** is one self-contained page published to a private claude.ai URL
- First publish asks permission; republishing the same artifact never asks again
- Every revision is a new **version** at the same URL; `Ctrl+]` reopens the latest
- Another session revises it only when you give it the URL



Sharing and styling artifacts

Audience, design tokens and page constraints

- Private by default; Pro and Max share a **public link**, Team and Enterprise share in the organization
- Design tokens in `CLAUDE.md` give the page your palette and typography
- Strict CSP: no external requests, no backend, a single page under 16 MiB
- Needs a `/login` session on the Anthropic API, not an API key or a cloud provider
- "Copy as prompt" brings the decision back into the session



Takeaways

What to carry into the harness module

- Route work to Haiku, Sonnet, Opus or Fable by task complexity
- Keep the prefix stable: no casual `/model` switches mid-session
- Enforce in `.claude/settings.json`, deny what you cannot undo
- Anchor checkpoints before risky work, isolate parallel work in worktrees
- Supervise background sessions from the agents view, share results as artifacts

